



MentorK.Tech

THE MENTORK.TECH BEGINNER SERIES · BOOK ONE

Where Do I Start?

A beginner's guide to finding your place in tech — the fields, the tools, the languages, and the first steps.

Software Development

AI & Machine Learning

Cybersecurity

Data Science

Cloud & DevOps

UI/UX

Front-End

Back-End

Networking & Hardware

Oladiti Kunle B.E (Mentork)

FOUNDER & CEO · MENTORK.TECH

LEARN IT · BUILD IT · LAUNCH IT



THE MENTORK.TECH BEGINNER SERIES · BOOK ONE

Where Do I Start?



A beginner's guide to finding your place in tech

MentorK.Tech
Technology · Creativity · Innovation

Where Do I Start?

A Beginner's Guide to Finding Your Place in Tech
The MentorK.Tech Beginner Series, Book One

Written by Oladiti Kunle B.E (MentorK), Founder & CEO of MentorK.Tech.
First edition, 2026. Published by MentorK.Tech.

Copyright © 2026 MentorK.Tech. All rights reserved. No part of this book may be reproduced, stored, or transmitted in any form without written permission from the publisher, except for brief quotations in reviews or for personal study.

Product names, tools, and platforms mentioned in this book are trademarks of their respective owners and are referenced for educational purposes only. Tool popularity and job-market details change quickly; always check current sources before making major decisions.

MentorK.Tech — We Train. We Mentor. We Build Futures.

Website: mentork-tech-website.vercel.app

WhatsApp: +234 705 777 9214 · +234 704 797 2890

Contents

	A Word from MentorK.Tech	2
	How to Use This Book	3
PART ONE — BEFORE YOU CHOOSE		
1	You Belong in Tech	4
2	The Map of Tech	7
PART TWO — THE FIELDS		
3	Software Development & Engineering	9
4	Artificial Intelligence & Machine Learning	15
5	Cybersecurity	21
6	Data Science	27
7	Cloud, Systems & DevOps	33
8	The Web Stack: UI/UX, Front-End & Back-End	39
9	Networking, Hardware & IT Support	44
PART THREE — CHOOSING AND STARTING		
10	The Master Comparison	48
11	Find Your Fit: A Self-Check	50
12	Your First 30 Days	52
13	Mistakes Beginners Make (and How to Avoid Them)	55
	Glossary: Words You Will Hear	58
	Learn It. Build It. Launch It. — with MentorK.Tech	61
	About the Founder	62

A Word from MentorK.Tech

If you have ever typed "how do I get into tech" into a search bar and closed the tab feeling more confused than when you opened it, this book was written for you.

Every week, people message us with the same question in different words. "I want to learn tech but I don't know where to start." "Should I do coding or design?" "Is data science the same as data analysis?" "Everyone is talking about AI — do I need maths?" "I've watched fifty videos and still haven't started anything."

The problem is rarely a lack of information. The real problem is that nobody hands beginners a **map**. Without a map, every path looks equally scary, and the easiest choice becomes no choice at all.

This book is that map. It walks you through the major fields in technology one by one, in plain English, and tells you honestly what people in each field do all day, which tools they open in the morning, which programming languages they speak, and how a complete beginner takes the first steps. At the end, you will compare the fields side by side, answer a short self-check, and leave with a 30-day starting plan.

At MentorK.Tech we train, we mentor, and we build. We have watched people who "were not computer people" become confident developers, designers, analysts, and network engineers. The difference was never talent. It was a clear starting point and someone to walk alongside them. Consider this book the starting point. We would be glad to be the ones walking alongside you.

The MentorK.Tech Team

We Train. We Mentor. We Build Futures.

How to Use This Book

Part One — Before You Choose

Two short chapters that clear away the myths ("I'm too old", "I'm bad at maths") and show you the whole landscape of tech in one picture.

Part Two — The Fields

Seven chapters, one per field, all following the same pattern:

- **In plain English** — what the field is, explained to someone with zero background.
- **What they actually do** — a realistic picture of the daily work and the job titles you will see.
- **Core tools, explained** — the software professionals use, and what it is for.
- **The languages** — which programming languages matter here and why.
- **Is this for you?** — a short checklist of signs that you would enjoy the field.
- **Beginner roadmap** — three stages from absolute zero to your first real project.

Part Three — Choosing and Starting

A one-page comparison of every field, a self-check to find your fit, a 30-day plan for any field, and the mistakes beginners make so you can skip them.

A SUGGESTION

Keep a notebook open while you read. Each time a sentence makes you think "that sounds like me", write it down with the chapter number. By the end of Part Two, your notebook will be pointing at your answer. Words in **bold** are defined in the Glossary at the back.

PART ONE

Before You Choose

Two things stop most beginners before they begin: believing they are not "the type", and not being able to see the whole picture. Let's fix both.

You Belong in Tech

Before we talk about fields and tools, we need to deal with the voice in your head that says tech is for other people.

Almost everyone we have mentored arrived carrying at least one of the myths below. None of them survived the first month of real learning. Read them, recognise yours, and let it go.

Myth 1: "I need to be a maths genius."

Most tech jobs use far less maths than people fear. A front-end developer, a UI/UX designer, a network engineer, or a cybersecurity analyst can go years without solving an equation. Only a few specialised areas, like machine learning research, lean heavily on mathematics — and even there, you learn it as you go, with tools doing the heavy lifting. What you do need is **logical thinking**: the ability to break a problem into steps. That is a skill, and skills are learned.

Myth 2: "I'm too old / too young / started too late."

People switch into tech at 17 and at 47. Employers and clients care about what you can build, not when you were born. A 35-year-old accountant who learns data analysis actually has an advantage: years of understanding what the numbers mean.

Myth 3: "I need an expensive laptop."

For almost every field in this book, a basic laptop that can run a web browser and a text editor is enough to start. Many tools run entirely inside the browser. You can learn UI/UX design, web development, data analysis, and networking basics on a modest machine. Upgrade later, when you know what you need.

Myth 4: "I need a computer science degree."

A degree helps, but the tech industry is unusually open to people who can show their work. A portfolio of real projects, a GitHub profile, or a well-made app will get you past doors that a certificate alone will not. Many of the best people in the industry are self-taught or mentor-taught.

Myth 5: "I'm not a computer person."

Nobody is born a computer person. Everybody who is one today was once staring at a screen, confused. Tech is not a personality type; it is a collection of learnable skills, and different fields reward very different personalities. Some are for people who love visuals. Some are for people who love puzzles. Some are for people who love order and systems. Some are for people who love helping others. There is room for you.

Myth 6: "I have to pick the perfect field now."

You do not. Fields in tech overlap enormously. A front-end developer who learns databases becomes a full-stack developer. A data analyst who learns Python drifts into data science. A network engineer who studies attacks becomes a security professional. Your first choice is a **starting point**, not a life sentence. The goal of this book is to help you choose a good starting point quickly, so you can begin, and then adjust as you learn what you enjoy.

Myth 7: "I watched the videos, so I'm learning."

Watching is not learning. Doing is learning. If you have watched ten hours of tutorials and built nothing, you have not started yet. The good news: the moment you build something tiny — a one-page website, a spreadsheet dashboard, a small script — you will learn more than in all those hours of watching combined.

WHAT YOU ACTUALLY NEED

- **Curiosity** — the willingness to ask "how does this work?"
- **Patience** — things break; fixing them is the job, not a sign of failure.
- **Consistency** — one hour every day beats ten hours once a month.
- **A starting point** — which is what the rest of this book gives you.

A promise before you turn the page

Every chapter ahead will speak to you plainly. When a technical word appears, it will be explained. When a tool is named, you will be told what it is for. You are not expected to know anything yet. That is the whole point.

The Map of Tech

"Tech" is not one job. It is a whole city with different neighbourhoods. Here is the city from above.

Think about what happens when you open a banking app on your phone and send money to a friend. In those few seconds, almost every field in this book was involved:

- A **UI/UX designer** decided where the buttons go and how the screens flow.
- A **front-end / mobile developer** built the screens you tap.
- A **back-end developer** wrote the code that checks your balance and moves the money.
- A **cloud engineer** keeps your account records safe and available 24/7.
- A **DevOps engineer** shipped the app update without breaking anything.
- A **cybersecurity analyst** is watching for fraud and attacks in the background.
- A **data scientist** built the model that flagged whether this transfer looks suspicious.
- A **network engineer** keeps your phone connected to the bank.
- An **AI engineer** trained the chatbot you talk to when something goes wrong.

Every one of these people is "in tech", yet most would struggle to do each other's jobs. That is why "learn tech" is not a plan — but "learn front-end development" is.

THE BIG IDEA

Tech fields are defined by **what problem they solve**, not by how hard they are. Pick the problem you find interesting, and the field follows.

The nine neighbourhoods

Software Development

Builds apps and programs for phones, desktops, and the web. *"Make the thing work."*

AI & Machine Learning

Teaches computers to learn from data and make decisions. *"Make the thing smart."*

Cybersecurity

Defends systems and data from attackers. *"Keep the thing safe."*

Data Science

Turns raw data into insights and predictions. *"Make the numbers talk."*

Cloud & DevOps

Runs and delivers software reliably at scale. *"Keep the thing running."*

UI/UX Design

Plans how products look and feel to use. *"Make the thing usable."*

Front-End Development

Codes the screens people see and touch. *"Build what the user sees."*

Back-End Development

Codes the logic, servers, and databases behind the screens. *"Build what the user doesn't see."*

Networking, Hardware & IT Support

Keeps computers, cables, and connections working. *"Keep the lights on."*

Where they meet

Fields overlap. Full-stack = front + back. MLOps = AI + DevOps. Your path can cross borders.

PART TWO

The Fields

Seven chapters, seven neighbourhoods. Each one explained from zero: what it is, what the work looks like, the tools, the languages, whether it fits you, and how to take the first steps.

As you read each one, ask yourself three questions:

Do I like the problem? Would I be happy thinking about this all day?

Do I like the work? Visual, logical, investigative, or systematic — which am I?

Can I start this week? Every field has a free entry point; the roadmap shows it.

03

Software Development & Engineering

"Make the thing work."

IN PLAIN ENGLISH

Software developers write the instructions that make computers, phones, and websites do useful things. Every app on your phone, every program on a laptop, every website you visit — someone wrote it, line by line. Software development is the craft of designing those instructions, writing them, testing them, and keeping them working as the world changes.

You'll enjoy it if	you like building things and seeing them work
Main languages	Python, JavaScript/TypeScript, Java, C#, C++, Kotlin, Swift
Signature tools	VS Code, Git & GitHub, Docker, Jira
Time to first project	2–4 weeks with daily practice

What they actually do

A software developer's job is to **translate a human need into working software**. Someone says "our customers should be able to book an appointment from their phone", and the developer turns that sentence into an app that does exactly that — reliably, quickly, and without losing anyone's data.

A typical day looks less like a movie hacker scene and more like this:

- **Reading and planning.** Understanding what needs to be built, and breaking it into small tasks.
- **Writing code.** Adding new features, one small piece at a time.
- **Testing.** Checking the code works — and, importantly, checking what happens when a user does something unexpected.
- **Fixing bugs.** A bug is a mistake in the code. Finding and fixing them is a large part of the job and is genuinely satisfying once you get the hang of it.
- **Reviewing.** Reading teammates' code and having yours read. This is how teams keep quality high and how juniors learn fastest.
- **Collaborating.** Short meetings with designers, testers, and product managers to stay aligned.

Job titles you will see

Title	What it means
Software Engineer / Developer	General builder of applications. The most common title.
Mobile Developer	Builds apps for Android (Kotlin/Java) or iPhone (Swift), or both with Flutter or React Native.
Full-Stack Developer	Comfortable on both the user-facing side and the server side of a web app.
Desktop / Systems Developer	Builds programs that run on Windows, Mac, or Linux, often in C++, C#, or Java.

Title	What it means
Game Developer	Builds games, usually with engines like Unity (C#) or Unreal (C++).
QA / Test Engineer	Specialises in testing software and writing automated tests. A common, friendly entry point.

Core tools, explained

Tools are just software that helps you make software. You will meet these four everywhere.

Tool	What it is for
VS Code, IntelliJ IDEA	An IDE (Integrated Development Environment) — the program you write code in. Think of it as Microsoft Word for code: it colours your text, spots typos, and helps you run your program. VS Code is free and the best place for beginners to start.
Git & GitHub	Git saves a history of every change to your code, so you can undo mistakes and work with others without overwriting each other. GitHub is the website where that history lives online — it is also where developers show off their work. Your GitHub profile becomes your portfolio.
Docker	Packages your program with everything it needs so it runs identically on any computer. It ends the classic problem of "but it works on my machine!" You will not need this in your first months, but you will hear the word constantly.
Jira, Trello	Project boards where teams list tasks, assign them, and move them from "To do" to "Done". Not technical — just how teams stay organised.

The languages

A **programming language** is the set of words and rules you use to give instructions to a computer. Different languages are popular for different kinds of software:

Language	Best known for
Python	Easiest to read. Great first language. Used everywhere from web back-ends to AI.
JavaScript / TypeScript	The language of the web. Every website runs it. TypeScript is JavaScript with extra safety checks.
Java	Large business systems, banking software, and Android apps. Very stable, very employable.
C#	Windows applications, business software, and Unity games. Similar feel to Java.
C++	When speed matters: game engines, operating systems, high-performance software. Harder; usually not a first language.
Kotlin / Swift	The modern languages for Android apps (Kotlin) and iPhone apps (Swift).

Beginner advice: start with **Python** (if you want general programming) or **JavaScript** (if you want to build things you can see in a browser quickly). Your second language will take a quarter of the time the first one did.

Is this for you?

Tick the ones that sound like you. Four or more is a strong sign.

- ☐ You enjoy building things and get a kick out of "it works!" moments.
- ☐ You like solving puzzles and don't mind being stuck for a while before it clicks.
- ☐ You are patient with small details — a missing comma can stop a whole program.
- ☐ You like the idea of creating apps or tools that other people actually use.
- ☐ You are happy to spend a lot of time reading, searching, and experimenting.
- ☐ You prefer clear right-or-wrong feedback (the program runs, or it doesn't).

Beginner roadmap

Stage 1 Foundations (weeks 1–4)

- Install **VS Code** and pick one language: Python or JavaScript.
- Learn the basics: variables, conditions (if/else), loops, functions, lists.
- Write tiny programs every day: a calculator, a guessing game, a to-do list in the terminal.
- Create a GitHub account and upload your tiny programs. Yes, even the ugly ones.

Stage 2 Building real things (months 2–4)

- Learn how programs read and save data (files, then a simple database).
- Learn to use other people's code: libraries and packages.
- Build one complete small project end-to-end: a budget tracker, a quiz app, a simple booking tool.
- Learn Git properly: commits, branches, and pull requests.

Stage 3 Specialise and show your work (months 4–8)

- Choose a direction: web, mobile, desktop, or games — and learn its main framework.
- Build two or three portfolio projects that solve real problems for real people (a local business, a church, a school, a friend's shop).
- Contribute a small fix to an open-source project on GitHub.
- Start applying for internships, junior roles, or freelance jobs.

FIRST PROJECT IDEA

A "Contact Me" web form for a real small business, that saves messages to a file or database. Small enough to finish, real enough to matter.

04

Artificial Intelligence & Machine Learning

"Make the thing smart."

IN PLAIN ENGLISH

Normal software follows rules a human wrote: "if the password is wrong, show an error." Machine learning is different — instead of writing the rules, you show the computer thousands of examples and let it *work out* the rules itself. That is how your phone recognises your face, how streaming apps guess what you'll like, and how chatbots write sentences. AI engineers are the people who gather those examples, train the systems, and put them to work.

You'll enjoy it if	you like patterns, experiments, and "what if" questions
Main languages	Python (the standard), R, C++, Julia
Signature tools	Jupyter, TensorFlow, PyTorch, Hugging Face, Scikit-learn
Time to first project	6–10 weeks (after Python basics)

What they actually do

An AI engineer's core loop is: **collect data** → **prepare it** → **train a model** → **test it** → **deploy it** → **improve it**. A **model** is simply the "learned" program that comes out of training; think of it as a very well-practised guesser.

In practice, the day is a mix of:

- **Working with data.** Gathering it, cleaning it (real data is messy: missing values, typos, duplicates), and labelling it. This is often 60–70% of the job, and it is where careful people shine.
- **Training models.** Choosing an approach, feeding data into it, and waiting — sometimes minutes, sometimes days.
- **Evaluating.** Measuring how often the model is right, and — crucially — how it fails. A model that is 95% accurate but always wrong for one group of people is not a good model.
- **Deploying.** Wrapping the model so an app or website can use it (this overlaps with back-end and DevOps skills).
- **Reading.** The field moves so fast that keeping up is part of the work.

Job titles you will see

Title	What it means
Machine Learning Engineer	Builds and deploys models into real products. Strong programming + ML knowledge.
AI Engineer	Often means building products <i>on top of</i> existing AI models (like large language models) using APIs. A very practical, fast-growing entry route.
Data Scientist	Overlaps heavily; usually more analysis and less deployment. See Chapter 6.
ML Researcher	Invents new methods. Usually needs advanced study and heavy mathematics.

Title	What it means
Prompt / AI Product Specialist	Designs how people and businesses interact with AI tools. Lighter on code, heavy on clear thinking.

HONEST NOTE ON MATHS

To *use* machine learning, you need comfortable basics: averages, percentages, simple graphs, and a willingness to learn a little statistics. To *invent* new machine learning, you need university-level maths. Almost all jobs are the first kind.

Core tools, explained

Tool	What it is for
Jupyter Notebooks	A workspace where you mix code, results, charts, and notes on one page — perfect for experimenting. Google Colab is a free version that runs in the browser on Google's computers, so you don't need a powerful laptop.
Scikit-learn	A Python library with ready-made "classic" machine learning methods. The friendliest place to train your first model.
TensorFlow, PyTorch	The two big libraries for deep learning — the technique behind image recognition, speech, and chatbots. PyTorch is currently the more beginner-friendly of the two.
Hugging Face	A website and library where thousands of pre-trained models are shared for free. You can download a model that already understands language or images and adapt it, instead of training from scratch.

Tool	What it is for
AI APIs (OpenAI, Anthropic, Google, etc.)	Let you use powerful models over the internet with a few lines of code. This is how most "AI-powered" apps are built today.
pandas, NumPy	Python libraries for handling tables of data and numbers. You will use them every single day.

The languages

Language	Why it matters here
Python	The absolute standard. Nearly every AI tool is built for Python first. Learn this and nothing else at the start.
R	Popular with statisticians and researchers. Useful, but optional.
C++	Used inside the high-performance engines that power AI. Not needed until much later, if ever.
Julia	A newer language for fast scientific computing. Niche; good to know it exists.
SQL	Not on most AI lists, but the language for pulling data out of databases — and you need data before you can train anything.

Beginner advice: do not start with AI. Start with Python. When you can write a program that reads a file and does something useful with it, you are ready for your first model.

Is this for you?

- ☐ You are curious about *why* things happen and enjoy spotting patterns.
- ☐ You like experiments: try, measure, adjust, try again.
- ☐ You are comfortable with uncertainty — models are "usually right", never "always right".
- ☐ You find it exciting, not scary, that the field changes every few months.
- ☐ You don't mind spending hours cleaning messy data to get a good result.
- ☐ You are willing to learn some statistics along the way.

Beginner roadmap

Stage 1 Python and data (weeks 1–6)

- Learn Python basics (see Chapter 3, Stage 1).
- Learn `pandas` : open a spreadsheet, filter rows, calculate averages, make a chart.
- Get comfortable in Google Colab or Jupyter.
- Learn the vocabulary: dataset, feature, label, training, testing, accuracy.

Stage 2 First models (months 2–4)

- Use `scikit-learn` to train classic models: predict house prices, classify emails as spam or not.
- Learn how to measure a model honestly (train/test split, accuracy, confusion matrix).
- Use an AI API to build a small app — a chatbot for a business, a summariser, a quiz generator.

Stage 3 Deep learning and deployment (months 4–9)

- Learn `PyTorch` basics and train an image classifier.
- Use a pre-trained model from Hugging Face and fine-tune it for a local problem (Nigerian food recognition, local-language text, market-price prediction).
- Put one model behind a simple web app so others can try it.
- Write up your projects clearly — in AI, explaining your results is half the job.

FIRST PROJECT IDEA

A WhatsApp-style FAQ bot for a small business, built on an AI API, that answers questions from a document you give it. Real, useful, and finishable in a weekend once you know Python.

05

Cybersecurity

"Keep the thing safe."

IN PLAIN ENGLISH

Every system that stores something valuable — money, personal details, company secrets — is a target. Cybersecurity is the practice of building walls around those systems, watching for anyone trying to climb over, and responding fast when someone does. Security professionals think like attackers so they can defend like professionals. Some of them are even paid to break into systems — with permission — to find the holes before criminals do.

You'll enjoy it if	you love investigating, and thinking "how would I get in?"
Main languages	Python, Bash, PowerShell, SQL, C/C++ (advanced)
Signature tools	Wireshark, Kali Linux, Nmap, Metasploit, Burp Suite, Splunk
Time to first project	4–8 weeks (networking basics first)

What they actually do

Security work splits into two broad sides, often called the **blue team** (defenders) and the **red team** (authorised attackers). Most beginners start blue.

- **Monitoring.** Watching network traffic and system logs for anything unusual: a login at 3 a.m. from another country, a computer suddenly sending huge amounts of data out.
- **Investigating.** When an alert fires, working out whether it is a real attack or a false alarm, and what the attacker touched.
- **Patching.** Keeping software updated and closing known weaknesses (**vulnerabilities**) before they are exploited.
- **Penetration testing.** With written permission, attempting to break into a company's systems the way a criminal would, then writing a report of what was found and how to fix it. This is "ethical hacking".
- **Setting up protections.** Firewalls, encryption, access rules, two-factor authentication.
- **Educating people.** Most breaches start with a human clicking a bad link. Training staff is real security work.

Job titles you will see

Title	What it means
Security Analyst / SOC Analyst	Monitors and responds to alerts in a Security Operations Centre. The most common entry-level role.
Penetration Tester	Authorised attacker. Finds and reports weaknesses.
Security Engineer	Builds and maintains the protective systems.
Incident Responder / Forensics	Investigates after a breach: what happened, how, and what was taken.

Title	What it means
GRC Analyst	Governance, Risk and Compliance — policies, audits, standards. Less technical, very in demand.

AN IMPORTANT LINE

The skills in this chapter are powerful. Using them on systems you do not own or have written permission to test is a crime, everywhere. Ethical hackers practise on purpose-built training platforms and labs. The whole profession is built on trust.

Core tools, explained

Tool	What it is for
Wireshark	Captures and shows every packet of data passing through a network, like a microscope for internet traffic. You can literally watch a website request travel.
Kali Linux	A version of the Linux operating system that comes pre-loaded with hundreds of security tools. The standard toolbox for penetration testers.
Nmap	Scans a network to discover which devices are connected and which "doors" (ports) they have open.
Metasploit	A framework for testing whether a known vulnerability can actually be exploited on a system — used in authorised tests.
Burp Suite	Sits between your browser and a website so you can inspect and modify what is being sent. The main tool for testing web application security.
Splunk (and other SIEMs)	Collects logs from thousands of systems into one searchable place and raises alerts. This is the screen a SOC analyst watches.

Tool	What it is for
Firewalls	Hardware or software that decides which traffic is allowed in and out of a network. The front gate.

The languages

Language	Why it matters here
Python	Automating repetitive security tasks, writing scanners, parsing logs, building small tools.
Bash / Shell	The command line of Linux. Security people live in the terminal; this is non-negotiable.
PowerShell	The command line of Windows. Essential because most companies run Windows.
SQL	Understanding databases lets you understand — and prevent — SQL injection, one of the most common attacks.
C / C++	For advanced work like analysing malware or finding low-level bugs. Not a starting point.

Beginner advice: before any tool, learn **how networks work** (IP addresses, ports, DNS, HTTP) and get comfortable in a Linux terminal. Security is applied networking plus applied curiosity.

Is this for you?

- ☐ You enjoy detective work — following clues to find out what really happened.
- ☐ When you see a system, you naturally wonder how it could be tricked or broken.
- ☐ You are careful, ethical, and comfortable with responsibility.
- ☐ You can stay calm under pressure (incidents happen at bad times).
- ☐ You like learning how things work under the hood — networks, operating systems.
- ☐ You are happy to keep learning forever; attackers never stop, so neither do defenders.

Beginner roadmap

Stage 1 Foundations (weeks 1–6)

- Learn networking basics: what an IP address is, what DNS does, how a web request travels.
- Install Linux (or use a virtual machine) and learn the terminal: navigating, files, permissions, users.
- Learn how operating systems, passwords, and encryption work at a basic level.
- Start Python basics for scripting.

Stage 2 Hands-on labs (months 2–5)

- Join a free legal practice platform (e.g. TryHackMe, Hack The Box, OverTheWire) and complete beginner paths.
- Use **Wireshark** to inspect your own traffic; use **Nmap** on your own home network.
- Learn the OWASP Top 10 — the ten most common web weaknesses — and test them on deliberately vulnerable practice apps.
- Consider a foundation certificate (e.g. CompTIA Security+) as a structured study path.

Stage 3 Specialise (months 5–12)

- Pick blue (SOC / defence) or red (penetration testing) and go deep.
- Build a home lab: a few virtual machines you attack and defend.
- Write up your lab exercises as reports — employers love seeing you can communicate findings.
- Enter beginner-friendly Capture-the-Flag (CTF) competitions.

FIRST PROJECT IDEA

A "security check-up" for a small business: review their Wi-Fi setup, password habits, software updates, and backups, then write a one-page plain-English report with fixes. No hacking required — and genuinely valuable.

06

Data Science

"Make the numbers talk."

IN PLAIN ENGLISH

Every business collects data — sales, clicks, deliveries, complaints, sensor readings — usually far more than anyone can read. Data science is the skill of turning those mountains of numbers into clear answers: Which products are really making money? Why did customers leave last month? What will sales look like in December? Data scientists clean the data, ask it questions, find the patterns, and explain them in charts and words that decision-makers can act on.

You'll enjoy it if	you like finding the story hidden in numbers
Main languages	Python, SQL, R (and Excel — seriously)
Signature tools	Excel, Power BI, Tableau, pandas, Jupyter, SQL Server, Spark
Time to first project	2–3 weeks (with Excel or SQL)

What they actually do

Data work is a ladder, and you can step on at any rung. At the bottom is **data analysis** — describing what happened. At the top is **data science** — predicting what will happen and recommending what to do. The daily work at all levels is:

- **Getting the data.** Pulling it from databases, spreadsheets, websites, or apps, usually with SQL.
- **Cleaning it.** Fixing dates written five different ways, removing duplicates, filling gaps. Unglamorous, essential, and where most of the time goes.
- **Exploring it.** Making quick charts and summaries to see what is going on.
- **Analysing and modelling.** Answering the actual question — from a simple comparison to a predictive model.
- **Communicating.** Building dashboards, writing reports, and presenting to people who do not care about code but care a great deal about the answer.

Job titles you will see

Title	What it means
Data Analyst	Describes and visualises what has happened. Excel, SQL, Power BI/Tableau. The friendliest entry point in all of tech for non-programmers.
Business Intelligence (BI) Analyst	Builds dashboards and reports that a company runs on. Very close to data analyst.
Data Scientist	Adds statistics and machine learning to predict and recommend. Python-heavy.
Data Engineer	Builds the pipelines that move and store data at scale. Overlaps with back-end and cloud.

Title	What it means
Analytics Engineer	Sits between engineer and analyst: makes data clean and ready for everyone else.

WHY MENTORK.TECH RECOMMENDS DATA ANALYSIS AS A FIRST STEP

You can start with tools you already have (Excel), see results on day one, and it uses knowledge from your current job. Many of our students who "hated coding" found their way into Python *through* data, because they finally had a reason to use it.

Core tools, explained

Tool	What it is for
Excel / Google Sheets	Still the most-used data tool on earth. Pivot tables, formulas, and charts take you surprisingly far. Learn it properly.
SQL databases (SQL Server, PostgreSQL, MySQL)	Where business data lives. You write SQL queries to ask questions like "total sales per region last quarter".
Power BI, Tableau	Drag-and-drop tools for building interactive dashboards that update automatically. What executives actually look at.
pandas, NumPy	Python libraries for working with tables of data. pandas is like Excel controlled by code — and it handles millions of rows.

Tool	What it is for
Jupyter Notebooks	The workspace for exploring data in Python: code, charts, and notes side by side.
Apache Spark, Hadoop	For "big data" — datasets too large for one computer. Advanced; you will not need them for a long while.

The languages

Language	Why it matters here
SQL	Essential. The language for asking databases questions. Learnable in weeks, used for a career.
Python	The main language for analysis beyond spreadsheets, and for all machine learning.
R	Built by statisticians for statistics. Strong in research and academia; optional elsewhere.
SAS	An older commercial tool still used in banks and healthcare. Learn only if a target employer uses it.
DAX / M (Power BI)	Formula languages inside Power BI. You will pick them up while building dashboards.

Beginner advice: **Excel** → **SQL** → **Power BI** → **Python**, in that order. Each step gives you a usable skill before the next one begins.

Is this for you?

- ☐ You like asking "why?" and are not satisfied with "that's just how it is".
- ☐ You are organised and notice when something in a table doesn't add up.
- ☐ You enjoy making things clear for other people — a good chart makes you happy.
- ☐ You are comfortable with basic maths (percentages, averages) and willing to learn some statistics.
- ☐ You already work with numbers in your job (sales, accounts, operations, research).
- ☐ You prefer solving business questions over building apps.

Beginner roadmap

Stage 1 Spreadsheets and SQL (weeks 1–5)

- Master Excel/Sheets: sorting, filtering, VLOOKUP/XLOOKUP, pivot tables, charts.
- Learn SQL: SELECT, WHERE, GROUP BY, JOIN. Practise on free sample databases.
- Find a real dataset you care about (local prices, football, elections, your own expenses) and answer three questions about it.

Stage 2 Dashboards and Python (months 2–4)

- Build an interactive dashboard in **Power BI** or **Tableau Public** (free).
- Learn Python basics, then **pandas** for cleaning and analysis, and a charting library.
- Learn basic statistics: distributions, correlation, and why averages can lie.

Stage 3 Prediction and portfolio (months 4–8)

- Train simple predictive models with `scikit-learn` (see Chapter 4).
- Complete three portfolio projects, each with a clear question, clean process, and a one-page summary a manager could read.
- Publish them on GitHub and share dashboards publicly.

FIRST PROJECT IDEA

Take a small business's sales records (even handwritten ones, typed into a sheet), clean them, and build a dashboard showing best-selling items, busiest days, and monthly trend. Present it to the owner. That is a real data analysis project.

07

Cloud, Systems & DevOps

"Keep the thing running."

IN PLAIN ENGLISH

When you use an app, the code is not running on your phone alone — it is talking to powerful computers called **servers** somewhere in the world. "The cloud" simply means renting those servers from companies like Amazon, Microsoft, or Google instead of buying your own. Cloud and DevOps professionals set up those servers, keep them running 24/7, make them scale when a million people show up at once, and build the automated conveyor belts that carry new code from a developer's laptop to live users safely.

You'll enjoy it if	you like systems, automation, and reliability
Main languages	Bash, Python, Go, YAML/JSON (configuration)
Signature tools	AWS, Azure, GCP, Terraform, Docker, Kubernetes, GitHub Actions
Time to first project	4–6 weeks (Linux basics first)

What they actually do

This chapter covers two closely related roles that share most of their tools.

Cloud Engineer / Architect

Designs and manages a company's computing footprint in the cloud. They decide which services to use, how to connect them securely, how to keep costs down, and how to make sure the system stays up. A common early project is **migration**: moving a company's old in-office servers into the cloud without anyone noticing.

DevOps Engineer

"DevOps" joins *Development* and *Operations*. Traditionally, developers wrote code and threw it over a wall to an operations team who tried to make it run. DevOps engineers remove the wall. They build **pipelines** — automated steps that test new code, package it, and release it, often many times a day — so that changes go live fast and safely.

Day to day, both roles involve:

- Writing configuration and scripts rather than app features. Much of the "code" is describing infrastructure.
- Monitoring dashboards: CPU, memory, errors, response times. Being on call when things break.
- Automating anything done more than twice.
- Security and cost reviews — a misconfigured cloud account can leak data or burn thousands of dollars in a night.

Job titles you will see

Title	What it means
Cloud Engineer	Builds and operates cloud infrastructure. Entry route via certifications.
Cloud Architect	Senior; designs whole systems. Years of experience.

Title	What it means
DevOps Engineer	Owns pipelines, automation, and deployments.
Site Reliability Engineer (SRE)	Focuses on uptime and performance; strong coding.
Systems Administrator	The classic role: manages servers, users, and backups. Great foundation.

Core tools, explained

Tool	What it is for
AWS, Microsoft Azure, Google Cloud (GCP)	The three big cloud providers, each offering hundreds of rentable services: servers, storage, databases, AI. Learn one deeply (AWS is most common); the others will feel familiar.
Linux	The operating system running most servers in the world. Not optional in this field.
Docker	Packages an application and its needs into a container that runs the same everywhere. The building block of modern deployment.
Kubernetes	Manages hundreds of containers: starts them, restarts crashed ones, spreads them across servers. Complex; learn after Docker.

Tool	What it is for
Terraform	Infrastructure as Code: you describe your servers and networks in a text file, and Terraform creates them. Repeatable, reviewable, no clicking around.
Jenkins, GitHub Actions	Pipeline tools. "When new code is pushed, run the tests, build the app, deploy it." GitHub Actions is the easier place to start.
Ansible	Configures many servers at once from one script.

The languages

Language	Why it matters here
Bash / Shell	The Linux command line. You will write small scripts constantly.
Python	Automation, cloud APIs, and glue between tools.
Go (Golang)	Many cloud tools (Docker, Kubernetes, Terraform) are written in Go. Useful at intermediate level.
YAML / JSON	Not programming languages but <i>configuration</i> formats. Pipelines, Kubernetes, and cloud settings are all written in them. You will read and write these daily.
Groovy	Used for Jenkins pipelines. Learn only if you use Jenkins.

Beginner advice: Linux terminal first, then one cloud provider's free tier, then Docker. Certifications (e.g. AWS Cloud Practitioner) give beginners a clear, employer-recognised path.

Is this for you?

- ☐ You like things to be orderly, reliable, and automated.
- ☐ You enjoy understanding how systems fit together more than designing screens.
- ☐ You are calm when something is on fire and can work through a checklist.
- ☐ You find it satisfying to make a slow, manual process fast and automatic.
- ☐ You are happy living in a terminal and reading documentation.
- ☐ You are comfortable with responsibility — real users depend on your systems.

Beginner roadmap

Stage 1 Linux and networking (weeks 1–6)

- Install Linux (or use a virtual machine) and live in the terminal: files, permissions, processes, services.
- Learn networking basics: IP, DNS, ports, HTTP, SSH.
- Write Bash scripts to automate small chores. Learn Git.

Stage 2 Cloud and containers (months 2–5)

- Open a free-tier account on AWS (or Azure/GCP). Launch a server, host a website on it, set up a database.
- Study for and take an entry-level cloud certification.
- Learn **Docker**: containerise a simple app you (or a friend) built.
- Build a **GitHub Actions** pipeline that tests and deploys that app automatically.

Stage 3 Infrastructure as code and scale (months 5–10)

- Recreate your cloud setup with `Terraform` .
- Learn `Kubernetes` basics and deploy a multi-container app.
- Add monitoring and alerts. Break things on purpose and practise recovering.
- Document everything in a public repository — your infrastructure projects are your portfolio.

FIRST PROJECT IDEA

Take a friend's website or portfolio, put it on a cloud server with a proper domain and HTTPS, and set it up so that every change they push to GitHub goes live automatically. That is DevOps in miniature.

08

The Web Stack: UI/UX, Front-End & Back-End

"Design it. Build what they see. Build what they don't."

IN PLAIN ENGLISH

Every website and web app is made of three layers built by three kinds of people. The **UI/UX designer** plans how it should look and feel, before any code. The **front-end developer** builds the part you see and click in your browser. The **back-end developer** builds the invisible engine behind it: the server, the logic, and the database that remembers everything. Together, they are the "web stack" — and this is the most popular starting point in tech, because you can see your results within days.

You'll enjoy it if	you want to build things people can use immediately
Main languages	HTML, CSS, JavaScript/TypeScript; Python, Node.js, Java, Go, C#
Signature tools	Figma, VS Code, React, Next.js, Tailwind, Node.js, PostgreSQL, Postman
Time to first project	1–2 weeks (a live one-page site)

Layer 1 — UI/UX Design

UI (User Interface) is how it looks: colours, layout, buttons, typography. **UX** (User Experience) is how it works for a person: can they find what they need, is the journey from "open app" to "done" smooth or frustrating? UI/UX designers research users, sketch flows, build clickable prototypes, and hand a finished design to developers.

What they do daily: interview or observe users; map out the screens and the path between them (a "user flow"); draw rough sketches (wireframes), then polished designs; test prototypes with real people; maintain a **design system** — the reusable buttons, colours, and components a product is built from.

Tool	What it is for
Figma	The industry standard. Design screens, build clickable prototypes, collaborate live. Free to start and runs in the browser.
Adobe XD, Sketch	Alternatives to Figma. Less common now, but you may meet them.
FigJam, Miro	Whiteboards for user flows, brainstorming, and research notes.

Programming languages: none. UI/UX is a visual and structural discipline. Knowing a little HTML/CSS helps you talk to developers, but it is not required. This is the best field in this book for creative people who do not want to code.

IS UI/UX FOR YOU?

- ☐ You notice when an app is confusing and can say exactly why.
- ☐ You care about how things look *and* how they work.
- ☐ You like talking to people and understanding their frustrations.
- ☐ You enjoy iterating: draft, feedback, improve, repeat.

Roadmap UI/UX

Weeks 1–4: learn design basics (layout, spacing, colour, type) and Figma. Redesign one screen of an app you use daily. **Months 2–4:** learn UX process — research, flows, wireframes, prototypes, usability tests. Design a complete small app. **Months 4–6:** build three case studies (problem → process → solution) into a portfolio site. Design a real product for a real client, even for free.

Layer 2 — Front-End Development

Front-end developers take the designer's screens and turn them into a working website or app that runs in a browser. They make it look right on a phone and a laptop, make buttons do things, and connect the screens to the back-end so real data appears.

What they do daily: build pages and components from designs; make layouts responsive (they adapt to any screen size); handle user actions (clicks, forms, scrolling); fetch data from the back-end and display it; test in different browsers; care about speed and accessibility so everyone can use the site.

Language / Tool	What it is for
HTML	The structure: headings, paragraphs, images, links, forms. Your first language — learnable in days.
CSS	The style: colours, fonts, spacing, layout, animation. Makes HTML look like a design.
JavaScript	The behaviour: what happens when you click, type, or scroll. The only programming language browsers understand.
TypeScript	JavaScript with type checking that catches mistakes early. Standard in professional teams; learn after JavaScript.
React, Next.js	A library (React) and a framework built on it (Next.js) for building complex, fast apps out of reusable components. The most in-demand front-end skills.

Language / Tool	What it is for
Tailwind CSS	A popular way to write CSS quickly using ready-made utility classes.
VS Code, Git, browser DevTools	Your editor, your history, and the built-in inspector in Chrome/Firefox that lets you see and tweak any website live.

IS FRONT-END FOR YOU?

- ☐ You want to see results instantly — change code, refresh, see it.
- ☐ You like the mix of visual work and logic.
- ☐ You care about the details of how things feel to use.

Roadmap Front-End

Weeks 1–4: HTML and CSS. Build and publish a one-page personal site (free hosting on GitHub Pages, Netlify or Vercel). **Months 2–4:** JavaScript fundamentals; make pages interactive; fetch data from a public API. **Months 4–7:** React, then Next.js and TypeScript; build two or three real sites for real people; make them fast and mobile-friendly.

Layer 3 — Back-End Development

Back-end developers build the engine room. When you log in, search, pay, or upload a photo, the front-end sends a request to the back-end, which checks it, does the work, saves or fetches data from a **database**, and sends the answer back. Users never see the back-end, but nothing works without it.

What they do daily: design and build **APIs** (the "menu" of requests an app can make); design databases; handle logins and security; integrate payments, SMS, and email; make things fast under heavy use; write tests.

Language / Tool	What it is for
Node.js (JavaScript)	Runs JavaScript on the server, so one language covers front and back. Very popular for startups.
Python (Django, FastAPI, Flask)	Readable and fast to build with. Great first back-end language.
Java, C#, Go, Ruby	Java/C# in large companies and banks; Go for high-performance services; Ruby (Rails) in many startups.
PostgreSQL, MySQL	Relational databases: data in tables, queried with SQL. Learn one; PostgreSQL is an excellent choice.
MongoDB, Redis	MongoDB stores flexible JSON-like records (common with Node.js); Redis is a super-fast in-memory store for caching.
Postman	A tool for sending test requests to your API and seeing the responses, without needing a front-end.

IS BACK-END FOR YOU?

- ☐ You care more about how things work than how they look.
- ☐ You like logic, structure, and data.
- ☐ You enjoy thinking about edge cases: "what if two people pay at the same second?"

Roadmap Back-End

Weeks 1–4: one language (Python or JavaScript). **Months 2–4:** build a simple API; connect a database; learn SQL; test with Postman. **Months 4–7:** add logins, uploads, and payments; deploy to the cloud; build a back-end for a real front-end. You are now **full-stack**.

09

Networking, Hardware & IT Support

"Keep the lights on."

IN PLAIN ENGLISH

Before any software can run, somebody has to make the physical world work: the computers, the cables, the Wi-Fi, the routers, the printers, the servers humming in the back room. Networking and hardware professionals build and maintain that physical and connective layer, and IT support specialists are the people every office calls when "the internet is down" or "my laptop won't start". It is the most hands-on field in this book, and one of the most reliable ways into tech without heavy programming.

You'll enjoy it if	you like fixing things with your hands and helping people directly
Main languages	Little to none at first; Bash/PowerShell and Python later
Signature tools	Cisco Packet Tracer, Wireshark, routers/switches, diagnostic tools
Time to first project	1–3 weeks (set up a small network or build a PC)

What they actually do

Network Engineer / Technician

Designs, installs, and maintains the networks that connect computers — inside an office, across a campus, or between cities. Configures routers and switches, sets up Wi-Fi, assigns addresses, secures the network, and troubleshoots when connections fail. Larger organisations, banks, ISPs, and universities all employ them.

Hardware / Computer Engineer

Works with the physical machines: assembling and upgrading computers, diagnosing faulty components, repairing laptops and phones, setting up servers, and in more advanced roles designing electronics and embedded systems (the small computers inside cars, appliances, and devices).

IT Support Specialist / Helpdesk

The front line. Sets up new staff, fixes software and hardware problems, manages user accounts, installs updates, and keeps everything working. It is the classic first job in tech and a launchpad into networking, security, cloud, or systems administration.

Core tools and skills

Tool / Skill	What it is for
Cisco Packet Tracer, GNS3	Free simulators where you build and configure virtual networks without buying equipment.
Routers, switches, access points	The hardware of networks. You learn to configure them through a command line.
Wireshark, ping, traceroute	Diagnostics: watch traffic and find where a connection breaks.
Windows / Linux administration	Managing users, installing software, fixing operating systems.

Tool / Skill	What it is for
Screwdrivers, multimeter, diagnostic software	For the hardware side. Yes, real tools.
Ticketing systems	Where support requests are logged and tracked (e.g. Jira Service Management, Zendesk).

Languages: you can start with none. Later, `Bash`, `PowerShell`, and `Python` let you automate network and support tasks — which is exactly how a technician becomes an engineer.

Is this for you?

- ☐ You like fixing things and feel good when something broken works again.
- ☐ You enjoy helping people face to face and explaining things patiently.
- ☐ You prefer hands-on work over sitting in code all day.
- ☐ You are methodical: check this, then this, then this, until you find the fault.
- ☐ You are curious about what is physically inside a computer or behind a Wi-Fi signal.
- ☐ You want a clear, certification-based path into employment.

Beginner roadmap

Stage 1 Fundamentals (weeks 1–6)

- Learn how computers work: components, operating systems, storage, memory.
- Learn networking basics: IP addresses, subnets, DNS, DHCP, routers vs switches, Wi-Fi.
- Practise in **Packet Tracer**: connect a few PCs and a router; make them talk.
- Take apart and reassemble an old PC or laptop.

Stage 2 Certifications and practice (months 2–6)

- Study for an entry certification: CompTIA A+ (hardware/support), Network+ or Cisco CCNA (networking).
- Set up a small real network: a home or shop with a router, a shared printer, and a file server.
- Learn Windows and Linux administration basics; start Bash/PowerShell scripting.

Stage 3 Specialise (months 6–12)

- Choose: network engineering, systems administration, security (Chapter 5), or cloud (Chapter 7).
- Take a helpdesk, technician, or internship role — experience compounds fast here.
- Document every network you build and every problem you solve; it becomes your portfolio.

FIRST PROJECT IDEA

Set up a small office or shop network from scratch: internet connection, secure Wi-Fi with a guest network, a shared printer, and simple backups. Draw the network diagram and write a one-page guide the owner can follow.

PART THREE

Choosing and Starting

You have seen the map. Now compare the fields side by side, check your fit, and take your first thirty days one step at a time.

The Master Comparison

Every field on one page. Use it to compare, then go back to the chapter that pulls at you.

Role	Daily mission	Must-know languages	Signature tools
Software Engineer	Build programs and apps	Python, JavaScript, Java, C#, Kotlin	VS Code, Git, Docker
AI / ML Engineer	Train systems to learn from data	Python (+ a little C++)	PyTorch, TensorFlow, Hugging Face
Cybersecurity Analyst	Stop attackers, patch weaknesses	Python, Bash, PowerShell	Kali Linux, Wireshark, Splunk
Data Analyst / Scientist	Find patterns and predict from data	SQL, Python, R	Excel, Power BI, Jupyter
Cloud Engineer	Design and run cloud systems	Python, Go, YAML	AWS / Azure / GCP, Terraform
DevOps Engineer	Automate testing and deployment	Bash, Python, Go	Docker, Kubernetes, GitHub Actions
UI/UX Designer	Design how products look and flow	None	Figma
Front-End Developer	Build what users see	HTML, CSS, JavaScript, TypeScript	React, Next.js, Tailwind
Back-End Developer	Build servers, logic, databases	Python, Node.js, Java, Go, SQL	PostgreSQL, Postman, Node.js

Role	Daily mission	Must-know languages	Signature tools
Network / IT Support	Keep computers and networks working	None at first; Bash/PowerShell later	Packet Tracer, routers, Wireshark

Quick guide by personality

If you are mostly...	Look first at
Visual and creative	UI/UX Design, Front-End Development
Logical and love building	Software Development, Back-End, Front-End
Curious about "why" and patterns	Data Analysis, Data Science, AI/ML
Investigative and careful	Cybersecurity
Systematic and reliability-minded	Cloud, DevOps, Networking
Hands-on and people-focused	IT Support, Hardware, Networking
Not a fan of coding	UI/UX, Data Analysis (Excel/Power BI), IT Support, Networking

Find Your Fit: A Self-Check

For each question, choose the answer that feels most like you — not the one you think sounds best. Write the letter down. There are no wrong answers.

1. A friend shows you a new app. Your first thought is:

- A. "The colours and layout could be so much better."
- B. "I wonder how they built this."
- C. "I wonder what data they're collecting and what it shows."
- D. "I wonder if this is secure."
- E. "I wonder what happens when a million people use it at once."
- F. "It won't connect — let me fix your Wi-Fi."

2. Which school subject did you (or would you) enjoy most?

- A. Art, design, or writing
- B. Building things — technical drawing, crafts, logic puzzles
- C. Mathematics, statistics, or economics
- D. History, law, or investigations
- E. Organising anything — you ran the club's schedule
- F. Physics, electronics, or fixing things at home

3. Your ideal work day is:

- A. Sketching, refining, and testing ideas with people
- B. Deep focus building something, seeing it work by evening
- C. Digging through information until the answer appears
- D. Chasing a mystery, piecing together what happened
- E. Making a messy process run itself automatically
- F. Moving around, hands on equipment, helping people directly

4. How do you feel about writing code?

- A. I'd rather not, or only a little
- B. Excited — that's the whole point
- C. Happy to, if it answers a question
- D. Fine, as a tool for a bigger goal
- E. Scripts and automation, yes; big apps, not so much
- F. Not yet — maybe later

5. When something breaks, you:

- A. Notice how confusing the error message is for the user
- B. Enjoy the challenge of tracing the bug in the code
- C. Look at the numbers to see when it started and why
- D. Ask whether someone did this on purpose
- E. Ask why the system didn't restart itself
- F. Check the cables, the power, the connection

6. Which result would make you proudest?

- A. "This app is so easy to use."
- B. "You built this yourself?"
- C. "Your report changed our decision."
- D. "You found the hole before the attackers did."
- E. "It hasn't gone down once all year."
- F. "Everything just works since you set it up."

Your result

Count your letters. Your most frequent letter is your first place to look; your second most frequent is your natural next step.

Mostly A UI/UX Design, then Front-End Development — Chapter 8.

Mostly B Software Development or Front-End / Back-End — Chapters 3 and 8.

Mostly C Data Analysis / Data Science, then AI & ML — Chapters 6 and 4.

Mostly D Cybersecurity — Chapter 5 (start with networking basics, Chapter 9).

Mostly E Cloud & DevOps — Chapter 7.

Mostly F Networking, Hardware & IT Support — Chapter 9.

A TIE? GOOD.

Ties usually mean a combined path: A+B is a front-end developer with design sense. C+B is a data engineer or ML engineer. D+F is a network security specialist. E+B is a DevOps engineer who can code. Your combination is your advantage.

Your First 30 Days

Whatever field you chose, the first month looks the same: set up, learn the basics, build one tiny thing, and show it to someone.

This plan assumes about one hour a day. If you have more time, go faster; if less, stretch it to 60 days. The order matters more than the speed.

Week 1 — Set up and commit

- 1 **Choose your field** using Chapter 11. Write it on paper and put it where you will see it.
- 2 **Install your tools.** One editor (VS Code, Figma, Excel, or Packet Tracer). Create a free GitHub account regardless of field.
- 3 **Pick one beginner course** — a single free, well-reviewed one. Not five. Not "the best one". One.
- 4 **Fix your schedule.** Same time every day. Tell someone you're doing it.

Week 2 — Learn the basics by doing

- 5 **Follow the course** — but type every example yourself. Never copy-paste.
- 6 **Break things on purpose.** Change a value; see what happens. This is how understanding grows.
- 7 **Keep a daily log:** what you learned, what confused you, one question to answer tomorrow.

Week 3 — Build one tiny thing

- 8 **Choose a project you can finish in a week.** Use the "First project idea" from your chapter, or something smaller.
- 9 **Finish it, even if it's ugly.** Done is the goal. Perfect is the enemy.
- 10 **Put it online** — GitHub, a Figma link, a published dashboard, a diagram and photos.

Week 4 — Show it and plan the next step

- 11 **Show your project to three people.** One who knows nothing about tech, one who knows a little, one who knows a lot. Listen to all three.
- 12 **Write a short post** about what you built and what you learned. It doesn't matter how many people read it. Writing it is what matters.
- 13 **Find a community** — a local meetup, an online group, or a mentorship programme like MentorK.Tech. Learning alone is the slowest way.
- 14 **Plan month two.** Look at Stage 2 of your chapter's roadmap and pick the next tiny project.

THE ONLY RULE

Do not restart. Beginners who fail usually don't fail at coding or design — they fail by switching courses, switching fields, or switching languages every time it gets hard. It *will* get hard around week two. That is normal. Keep going for the full thirty days before you judge anything.

What "progress" really looks like

New learners expect a smooth upward line. Real learning looks like this:

- **Days 1–5:** Excitement. Everything is new and it feels easy.
- **Days 6–15:** The dip. Concepts pile up, errors multiply, and you wonder if you're "cut out for it". Everyone feels this. Everyone.
- **Days 16–25:** The first click. Something that was confusing suddenly makes sense. You start to fix your own errors.
- **Days 26–30:** Momentum. You finish something. You want to build the next thing.

If you know the dip is coming, it loses its power. Push through it once, and you will never be a beginner in the same way again.

THIRTY DAYS FROM NOW

You will have: a chosen field, working tools, one finished project online, a log of what you learned, and a plan for month two. That is more than most people who "want to get into tech" ever achieve. Start today.

Mistakes Beginners Make

We have mentored enough beginners to know exactly where they trip. Here are the traps, and how to step around them.

1. Tutorial hell

Watching course after course without building anything. It feels productive and produces nothing. **Fix:** the moment you learn a concept, use it in a project of your own, however small.

2. Tool-hopping

"Should I learn React or Vue? Python or JavaScript? AWS or Azure?" — asked for months while learning none of them. **Fix:** the first choice barely matters. Skills transfer. Pick the one in this book and move.

3. Learning in secret

Waiting until you are "good enough" before showing anyone. You never feel good enough, so you never share, so you never get feedback, so you never improve. **Fix:** share from week one. A GitHub with ugly beginner code is worth more than a perfect one that doesn't exist.

4. Copy-pasting instead of typing

Your fingers learn alongside your brain. Copy-paste teaches neither. **Fix:** type everything, even when it's slow.

5. Skipping the fundamentals

Jumping to React before JavaScript, to machine learning before Python, to Kali Linux before networking. It works until it very suddenly doesn't. **Fix:** follow Stage 1 of your roadmap even when it feels basic.

6. Comparing your day 10 to someone's year 5

Social media is full of people showing their finished work, never their struggle. **Fix:** compare yourself only to yourself last month.

7. Buying gear instead of building skill

A new laptop, a mechanical keyboard, three monitors — none of it makes you learn faster. **Fix:** use what you have until it genuinely blocks you. Then upgrade the one thing that's blocking you.

8. Fearing error messages

Beginners see red text and panic. Professionals see red text and read it — because the error message usually says exactly what's wrong and where. **Fix:** read the error slowly. Search the exact message online. Nine times out of ten, someone has hit it before you.

9. Not asking for help — or asking badly

Struggling alone for days is not heroic; it is slow. But "it doesn't work, please help" gets no answers. **Fix:** ask early, and ask well: what you're trying to do, what you tried, what happened, and the exact error. You will often solve it yourself while writing the question.

10. Giving up in the dip

Around week two or three, almost everyone hits a wall and concludes they're "not a tech person". This is the single biggest reason beginners quit — and it is the most predictable, most temporary stage of all. **Fix:** re-read Chapter 12. Expect the dip. Keep your daily hour. Get a mentor or a study partner who has been through it and can tell you it passes.

THE PATTERN BEHIND EVERY MISTAKE

Look at the list again. Almost every trap is a way of **avoiding the discomfort of building something real**. Watching, choosing, buying, hiding, waiting — all of them feel like progress and cost nothing. Building feels uncomfortable and costs effort. That is exactly why it works.

ONE SENTENCE TO REMEMBER

The person who builds one small, imperfect thing every week will, in a year, be unrecognisable — and will be far ahead of the person still choosing the perfect course.

Glossary: Words You Will Hear

Algorithm

A step-by-step recipe for solving a problem. Every program is made of them.

API (Application Programming Interface)

A menu of requests one program can make to another. Your weather app uses an API to ask a server for today's forecast.

Back-end

The server-side part of an app: logic, databases, security. Users don't see it.

Bug

A mistake in code that makes a program behave wrongly. Fixing them is called debugging.

Cloud

Renting computing power and storage from providers like AWS, Azure, or Google over the internet, instead of owning servers.

Container (Docker)

A package holding an app and everything it needs, so it runs the same anywhere.

Database

Organised storage for data that programs can read and write. SQL is the common language for asking it questions.

Deploy

To put software live where real users can reach it.

DevOps

The practice (and role) of automating how code is tested and released, joining development and operations.

Framework

A ready-made structure for building a certain kind of app quickly (React for interfaces, Django for back-ends).

Front-end

The part of a website or app that runs in the user's browser or on their screen.

Full-stack

A developer comfortable with both front-end and back-end work.

Git / GitHub

Git tracks every change to your code; GitHub hosts it online and doubles as a portfolio.

IDE

Integrated Development Environment — the program you write code in, like VS Code.

IP address

The unique number identifying a device on a network, like a postal address for computers.

Library / Package

Code someone else wrote that you can reuse, so you don't rebuild common things from scratch.

Linux

A free operating system that runs most servers and is the home of security and DevOps work.

Machine learning model

A program that has "learned" patterns from example data and can make predictions on new data.

Open source

Software whose code is public and free to use, study, and improve. Contributing to it is a great way to learn.

Pipeline (CI/CD)

An automated sequence that tests, builds, and releases code whenever it changes.

Port

A numbered "door" on a device through which a particular kind of network traffic passes.

Portfolio

A collection of your real projects that shows employers and clients what you can do. More powerful than any certificate.

Prototype

A rough, clickable version of a design used to test ideas before building.

Responsive design

A website that adapts its layout to phones, tablets, and desktops.

Server

A computer whose job is to serve data or services to other computers, usually running 24/7.

SQL

Structured Query Language — the standard language for asking databases questions.

Terminal / Command line

A text-based way of controlling a computer by typing commands. Essential in most technical fields.

UI / UX

User Interface (how it looks) and User Experience (how it feels to use).

Version control

Tracking changes to files over time so you can go back, compare, and collaborate. Git is the standard.

Virtual machine (VM)

A computer running inside another computer as software. Lets you try Linux or build a lab without extra hardware.

Vulnerability

A weakness in software or a system that an attacker could exploit.

Wireframe

A simple sketch of a screen's layout, before colours and details are added.

Learn It. Build It. Launch It.

You now have a map, a compass, and a 30-day plan. The one thing this book cannot give you is someone walking beside you. That is what MentorK.Tech does.

Learn & grow with us

We train and mentor beginners in:

UI/UX Design

Front-End Development

Back-End Development

Data Analysis

Computer Networking

Hardware Engineering

Modern Programming & Web Technologies

...and more

From absolute zero or stuck halfway, our mentors help you choose a field, follow a roadmap, and build a portfolio that opens doors.

Need a tech solution? We build it.

We also deliver professional technology services for individuals, businesses, and startups:

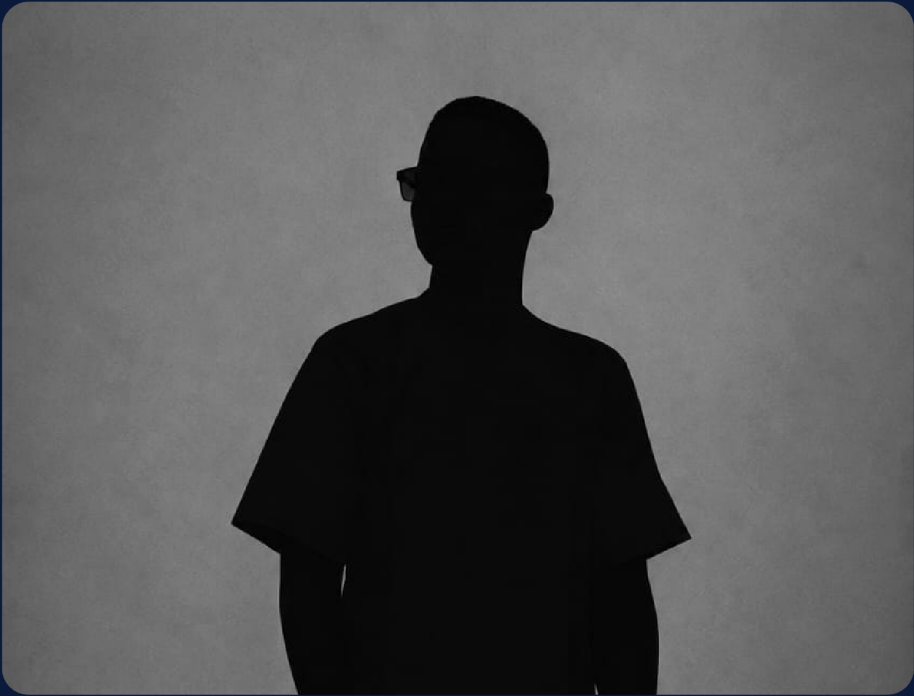
- Portfolio & personal websites
- Company & business websites
- Mobile application development
- Web applications
- UI/UX design
- Backend systems & APIs
- Data & digital solutions
- Technical & IT solutions

YOUR IDEA. OUR TECH. LET'S BUILD IT — MESSAGE US ON WHATSAPP

+234 705 777 9214 · +234 704 797 2890

Tell us: your name, the field you chose from this book, and what you want to build. We'll take it from there.

See our work and programmes: mentork-tech-website.vercel.app



ABOUT THE FOUNDER

Oladiti Kunle B.E (Mentork)

Founder & CEO, MentorK.Tech

MentorK.Tech was founded on a simple belief: the gap between people who "want to get into tech" and people who are building with it is not talent. It is direction, and someone willing to walk alongside you.

Kunle began, like the reader of this book, with more questions than answers — and built MentorK.Tech to be the guide that was missing then. Today the company trains and mentors beginners across design, development, data, networking, and hardware, and builds digital solutions for businesses and organisations that need their ideas turned into working products.

This book is the first in a series written to do at scale what the company does one person at a time: make the start clear, make the path real, and make the future feel buildable.

Innovate. Empower. Excel. mentork-tech-website.vercel.app

Everyone says "learn tech". Nobody says where to begin.

Software or design? Data or security? Cloud, networking, AI? To a beginner, every path looks the same — equally exciting, equally impossible. So most people never start.

This book is the map that was missing. In plain English, it walks you through the nine major fields of technology: what each one really is, what the work looks like day to day, which tools and languages matter, whether it fits your personality, and exactly how to take your first steps. It ends with a side-by-side comparison, a self-check to find your fit, and a 30-day plan you can start tonight.

You do not need a degree, a powerful laptop, or a head for maths. You need a starting point. This is it.

We Train

Beginner mentorship in UI/UX, web development, data, networking, hardware, and more.

We Mentor

Structured roadmaps, real projects, and someone to walk beside you.

We Build

Websites, mobile and web apps, APIs, data and IT solutions for businesses and startups.

We Launch

Your idea, turned into a working digital product. Tell us what you have in mind.

MentorK.Tech

mentork-tech-website.vercel.app

WhatsApp: +234 705 777 9214 · +234 704 797 2890

WE TRAIN. WE MENTOR. WE BUILD FUTURES.

